Team rehearsal and presentation plan

Group 15 • Final assessment preparation • 3 November 2026

The group has complementary backgrounds in Computing Science, Data Science, and Business Informatics. Use those strengths to improve explanations, but train every member to answer across the whole project. The rubric explicitly evaluates individual understanding and permits redirected questions.

The speaking roles below are proposed rehearsal responsibilities, not a historical division of implementation work. Record actual contributions separately and truthfully.

A common standard for every member

Before the exam, each person should be able to do all six tasks without notes:

1. Draw the five application responsibilities and identify PostgreSQL as the authority.
2. Follow one measurement from Avro decoding through complete-sensor averaging to durable history and notification.
3. Explain the database/Kafka acknowledgement gap and the distinct external-notification duplicate window.
4. State the successful workload and results with correct units, dates, and scope.
5. Diagnose at least two different bottlenecks and select different responses for them.
6. Explain the current recovery status, a limitation, and a proposed improvement without claiming it is already implemented.

A member who can only explain “their” component is not ready under this rubric.

Use the three backgrounds productively

Preparation lead	Lead topic	Cross-training obligation
Computing Science	Ordering, optimistic concurrency, atomic transactions, failure cases, safe scaling	Explain experimental limitations and costs in accessible language
Data Science	Workload design, offered versus achieved rate, percentiles/maxima, before/after inference, repeatability	Walk through checkpoint and outbox code
Business Informatics	Requirement traceability, experiment value, operational risk, cost scope, incident/recovery decisions	Calculate a sensor example and explain cache/version correctness

Each lead should teach the other two, who then explain the topic back using a new example. Correct the mechanism, not just the terminology.

Four preparation sessions

Session 1 Shared mental model

Spend 45–60 minutes on the architecture and worked sensor example. Each person draws the system independently; compare ownership, public interfaces, and data flow. Introduce a duplicate sensor, a missing sensor, and a phase change with incomplete input.

Finish by asking a different member to explain why the outside-history count differs from the notification count. Success means all three produce the correct outputs, not merely agree with the lead.

Session 2 Failure and code walkthrough

Spend 60 minutes opening the actual files. Rotate explanation of engine.apply, executor.prepare_commit, scheduler.dispatch_partition, Data.commit, Data.claim/result, and notifier.deliver.

Inject failures verbally before commit, after commit but before Kafka acknowledgement, and after external acceptance. Then explain the stale-middle-version regression and why it protects the performance optimization.

Success means each member can identify both the relevant function and the invariant it protects.

Session 3 Evidence and bottlenecks

Spend 45–60 minutes reading the raw JSON results. Ask one member to state the workload, another to calculate a derived quantity, and the third to challenge the claim.

Use these prompts: Why did 632 become 610? Which run has HTTP p95 0.689 seconds? What does 202 count? Why is 97.310/s different from a target-rate metric of 100? Why do old failing profiles need a rerun after optimization?

Finish with the three bottleneck windows in the oral handbook. Success means distinguishing measured observations from plausible explanations and future tests.

Session 4 Mock assessment

Use a ten-minute practice presentation followed by twenty minutes of randomly redirected questions. Ten minutes is a rehearsal format, not a confirmed exam duration. Shorten or expand the modules to the professor's allotted time.

One member acts as examiner, one answers, and one checks correctness/evidence. Rotate after each question. Repeat once so no member remains the examiner for all technical questions.

A ten-minute presentation outline

Time	Speaker for this rehearsal	Message and visual
0:00–1:00	Business Informatics	Problem, monitoring-only scope, history and timely notification, rubric targets
1:00–3:00	Computing Science	Architecture diagram, responsibility boundaries, one complete measurement
3:00–4:30	Computing Science	Atomic checkpoint/state/history/outbox; one crash example
4:30–6:30	Data Science	Exact workload, failed-to-passing comparison, what the improvement changed
6:30–8:00	Data Science	Replica matrix and bottleneck diagnosis; why more workers did not always help
8:00–9:15	Business Informatics	Storage incident, recovery, cost scope, operational limits
9:15–10:00	Any member	Demonstrated result, remaining variable-load evidence, next targeted step

Prepare eight simple slides or views corresponding to these topics. Use the generated architecture and performance figures. Show a raw result or code function when it helps answer a specific question; do not scroll the complete code appendix during the main explanation.

Suggested opening

“We monitor experiments by averaging all allocated sensors for each sampling instant, retaining carry-out history, and notifying required temperature transitions. We separated the ingestion, computation, storage, query, and delivery responsibilities. Our design uses a database transaction to keep state, history, notification obligations, and Kafka progress consistent. We then measured the design, found a database overhead problem, and improved it while preserving the consistency checks.”

Suggested performance explanation

“The first full instructor-tool test had correct history but three notifications above ten seconds and insufficient achieved HTTP rate. We reduced sequential database round trips by pipelining state writes, with all version checks inside one transaction. The next run, under the same settings, received all 610 required notifications with a 5.641-second maximum and 97.310 successful HTTP requests per second. The simulator is random, so this is an observed improvement rather than a controlled repeated causal estimate.”

Suggested limitation explanation

“Scalability depends on the limiting resource. Our worker abstraction can scale, but extra workers on a saturated shared VM competed with reads, and the Data process's local metadata cache currently prevents casual replication. We would measure changing workload on the final implementation and address the observed bottleneck first. We also need operational storage management because a full disk can stop every service on this host.”

If the deployment has not yet been restored, say so directly. An old screenshot is historical evidence, not a substitute for a working live demo.

Random redirection drills

Ask these in a different order every round:

- Business Informatics member: why can external notification delivery be duplicated?
- Data Science member: which four categories of effect commit with the checkpoint?
- Computing Science member: why does one before/after pair not isolate a causal speedup?
- Business Informatics member: what makes multiple Data replicas unsafe now?
- Computing Science member: why is the cost metric not the provider's bill?
- Data Science member: what happens if the last sensor of an old group arrives after a newer group is complete?
- Any member: what can an empty green dashboard establish?
- Any member: what is the first action when CPU is low but outbox age rises?
- Any member: what did the Docker commands remove, and what is still unverified?
- Any member: what evidence is missing to support Excellent under variable HTTP traffic?

The observer should record the mistaken mechanism or unsupported claim, then point to the relevant function or artifact. Re-answer after five minutes using a different concrete example.

Self-assessment without inventing a grade

Use this preparation scale for each topic: 0 = cannot explain; 1 = repeats terms but cannot apply them; 2 = correct explanation plus worked example; 3 = explains mechanism, evidence, tradeoff, and counterexample.

Topic	Computing Science	Data Science	Business Informatics
Requirements and complete-sensor averaging	Unassessed	Unassessed	Unassessed
Kafka ordering and replay	Unassessed	Unassessed	Unassessed
Transactions and optimistic concurrency	Unassessed	Unassessed	Unassessed
Outbox, leases, and external duplicates	Unassessed	Unassessed	Unassessed
Cache correctness and Data scaling	Unassessed	Unassessed	Unassessed
Measurement and experimental limits	Unassessed	Unassessed	Unassessed
Bottleneck diagnosis	Unassessed	Unassessed	Unassessed
Demo recovery and costs	Unassessed	Unassessed	Unassessed

This is a rehearsal tool, not the professor's numeric grade. Aim for no topic below 2 for any member and level 3 on each member's lead topic.

Actual contribution record

Before submission, fill a factual record with member name, concrete task, artifact/commit/result, decision made, verification, and what the member can demonstrate. Include code, experiments, analysis, operations, and documentation where they happened. Use dates when supported by the record.

If a contribution cannot be substantiated, describe the actual learning or review work rather than manufacturing authorship. A good oral answer shows understanding of how an artifact was produced and validated.

Responding when uncertain

Take a moment, identify the assumption, and reason from the implementation. For example: “The worker is stateless with respect to experiment ownership; the partial group is in Data. I would check the Data commit result and checkpoint before deciding whether replay is required.”

If the source or evidence does not settle a question, say what is known and what you would inspect. “The matrix shows a throughput loss with more replicas; it does not isolate database wait from CPU competition” is stronger than inventing a profiler result.

The objective is not memorizing all fifty answers. It is being able to derive the right answer from requirements, state ownership, failure boundaries, and evidence.